



waterTest 项目说明

1. 项目简介

`waterTest` 是一个基于 ESP32 的水冷监测固件，主要完成以下功能：

- 读取 MAX31855 热电偶温度
- 检测漏水状态
- 使用 u8g2 驱动 OLED 显示屏
- 通过 Blinker App 上报温度、阈值和漏水状态
- 支持温度阈值、WiFi 配置和 Blinker auth 保存到 ESP32 NVS / Preferences
- 支持漏水或超温时触发蜂鸣器报警
- 支持通过串口配置 WiFi SSID、WiFi 密码和 Blinker auth

该项目的核心文件是 [watertest.ino](https://github.com/foobaz/watertest.ino)。

2. 硬件组成

2.1 主控

- ESP32

2.2 传感器与外设

- MAX31855 热电偶模块
- 漏水检测模块
- OLED 128x64 I2C 显示屏
- 蜂鸣器

2.3 默认引脚

功能	引脚
MAX31855 SCK	GPIO 13

功能	引脚
MAX31855 CS	GPIO 12
MAX31855 DO	GPIO 14
漏水输入	GPIO 25
蜂鸣器	GPIO 2
OLED SDA	GPIO 33
OLED SCL	GPIO 32

3. 功能说明

3.1 温度采集

固件会读取 MAX31855 的热电偶温度和冷端温度，并对热端温度做线性拟合后显示和上报。

当前采用的两点拟合标定是：

- 显示 60°C 时，实际约 23°C
- 显示 230°C 时，实际约 90°C

对应换算公式为：

$$T_{real} = 0.3941176 \times T_{display} - 0.6470588$$

3.2 漏水检测

漏水输入默认按低电平触发，检测到漏水后会：

- OLED 显示报警画面
- 向 Blinker App 发送漏水提示
- 触发蜂鸣器慢响

3.3 温度阈值

阈值默认值为 60.0°C，可通过 Blinker App 的数值控件 `ran-bdm` 修改，也可以在 App 端继续保

持同步。

阈值会写入 NVS / Preferences，掉电后仍然保留。

3.4 App 上报

App 中使用了三个数值控件：

- `num-tem`：热端温度
- `num-hhp`：冷端温度
- `ran-bdm`：温度阈值

另外还会上报漏水状态，并在超温或漏水时发送通知消息。

3.5 蜂鸣器报警

蜂鸣器接在 GPIO 2 上，报警方式是慢速间歇鸣叫。

- 漏水报警：慢响
- 超温报警：慢响
- App 按钮 `btn-bs` 可以消音当前报警
- 当下一次新的报警条件出现时，会自动重新响起

3.6 OLED 显示

显示部分使用 u8g2 库，支持：

- 启动画面
- 正常数据画面
- 全屏告警画面

4. 串口配置流程

固件支持通过串口设置 WiFi SSID、WiFi 密码和 Blinker auth。串口波特率为 `115200`，行尾选择 `CRLF`。

4.1 串口启动后的默认提示

上电后，串口会输出帮助信息和当前配置，方便确认当前状态。

4.2 可用命令

查看帮助

```
help
```

输出完整命令说明。

查看当前配置

```
show
```

输出当前保存的：

- WiFi SSID
- WiFi 密码
- Blinker auth
- 温度阈值

同时设置 WiFi 和 Blinker auth

```
cfg <ssid> <password> <blinkerauth>
```

示例：

```
cfg MyWiFi 12345678 abcdef123456
```

只设置 WiFi

```
set wifi <ssid> <password>
```

示例：

```
set wifi MyWiFi 12345678
```

只设置 Blinker auth

```
set auth <blinkerauth>
```

示例：

```
set auth abcdef123456
```

一次性设置全部

```
set all <ssid> <password> <blinkerauth>
```

示例：

```
set all MyWiFi 12345678 abcdef123456
```

4.3 带空格参数的写法

如果参数中包含空格，可以使用双引号包起来：

```
set wifi "My Home WiFi" "12345678"
```

4.4 保存与重启行为

当你使用 `cfg`、`set wifi`、`set auth` 或 `set all` 成功修改配置后，固件会：

1. 保存配置到 NVS / Preferences
2. 打印当前配置
3. 自动重启
4. 使用新配置重新连接 WiFi 和 Blinker

保存成功后，固件还会立即从 NVS 回读一次并比对，确认数据真实写入。

5. WiFi 和 Blinker auth 设置流程

这是最常用的配置流程，建议按下面步骤操作。

步骤 1：连接串口

把 ESP32 连接到电脑后，打开串口监视器，波特率设置为 `115200`。

步骤 2：查看帮助

输入：

```
help
```

确认命令是否正常输出。

步骤 3：查看当前配置

输入：

```
show
```

确认当前保存的 WiFi SSID、WiFi 密码和 Blinker auth。

步骤 4：修改 WiFi 和 Blinker auth

建议使用以下任一方式：

```
cfg <ssid> <password> <blinkerauth>
```

或者：

```
set all <ssid> <password> <blinkerauth>
```

例如：

```
cfg HomeWiFi 12345678 156e262d8e93
```

步骤 5：等待自动重启

保存成功后设备会自动重启，重启后会使用新配置重新连接网络。

步骤 6：确认连接状态

重启完成后，观察：

- 串口输出是否正常
- Blinker App 是否在线
- `num-tem`、`num-hhp`、`ran-bdm` 是否有数据刷新

如果串口打印出 NVS raw read on load，可直接查看回读出来的 WiFi SSID、Blinker auth 和阈值，判断是否真的保存成功。

6. App 侧配置说明

Blinker App 中至少需要准备以下控件：

- 数值控件 `num-tem`
- 数值控件 `num-hhp`
- 数值控件 `ran-bdm`
- 按钮控件 `btn-bs`

6.1 数值控件

- `num-tem` 用于显示热端温度
- `num-hhp` 用于显示冷端温度
- `ran-bdm` 用于显示和调节阈值

6.2 按钮控件

- `btn-bs` 用于消音当前报警

7. 运行逻辑

固件的基本运行顺序如下：

1. 启动串口
2. 从 NVS / Preferences 读取温度阈值和网络配置
3. 用读取到的 WiFi SSID、WiFi 密码和 Blinker auth 初始化连接
4. 初始化漏水输入、蜂鸣器和 OLED
5. 进入循环后持续读取串口命令
6. 定时采集温度和漏水状态
7. 每秒向 App 上传数据
8. 超温或漏水时发送通知并触发蜂鸣器

8. 常见问题

8.1 App 连不上

优先检查：

- WiFi SSID 是否正确
- WiFi 密码是否正确
- Blinker auth 是否正确
- 手机和 ESP32 是否在同一可用网络环境中

8.2 App 没有显示数值

优先检查：

- App 里的控件类型是否为数值控件
- 控件 key 是否和固件一致：`num-tem`、`num-hhp`、`ran-bdm`
- 是否已经成功连接到 Blinker

8.3 蜂鸣器不响

优先检查：

- 蜂鸣器是否接在 GPIO 2

- 报警条件是否真的成立
- 是否已经被 `btn-bs` 消音

8.4 漏水或超温提示不出现

优先检查：

- 漏水输入电平是否和 `WATER_ACTIVE_LEVEL` 一致
- 温度补偿是否适合你的硬件
- 阈值是否设置过高

9. 代码中的关键配置项

- `DEFAULT_TEMP_THRESHOLD_C`：默认温度阈值
- `HOT_TEMP_FIT_SLOPE`：热端线性拟合斜率
- `HOT_TEMP_FIT_INTERCEPT`：热端线性拟合截距
- `SAMPLE_INTERVAL_MS`：采样周期
- `BLINKER_UPLOAD_INTERVAL_MS`：上报周期
- `BLINKER_AUTH_MAX_LEN`：Blinker auth 最大长度
- `WIFI_SSID_MAX_LEN`：WiFi SSID 最大长度
- `WIFI_PSWD_MAX_LEN`：WiFi 密码最大长度

10. 建议的使用顺序

1. 先写好硬件连接
2. 再通过串口设置 WiFi 和 Blinker auth
3. 然后在 App 中创建对应控件
4. 最后观察 OLED、串口和 App 的同步状态

11. 备注

当前固件内置了默认配置，仅用于首次启动或 NVS 未初始化时使用。实际部署时，建议通过串口重新设置成自己的 WiFi 和 Blinker 信息。